



Practica 3

[Practica-3---Monitores.pdf](#)

CONSIDERACIONES PARA RESOLVER LOS EJERCICIOS:

- Los monitores utilizan el protocolo signal and continue.
- A una variable condition SÓLO pueden aplicársele las operaciones SIGNAL, SIGNALALL y WAIT.
- NO puede utilizarse el wait con prioridades.
- NO se puede utilizar ninguna operación que determine la cantidad de procesos encolados en una variable condition o si está vacía.
- La única forma de comunicar datos entre monitores o entre un proceso y un monitor es por medio de invocaciones al procedimiento del monitor del cual se quieren obtener (o enviar) los datos.
- No existen variables globales.
- En todos los ejercicios debe maximizarse la concurrencia.
- En todos los ejercicios debe aprovecharse al máximo la característica de exclusión mutua que brindan los monitores.
- Debe evitarse hacer busy waiting.
- En todos los ejercicios el tiempo debe representarse con la función delay.

-
1. **Se dispone de un puente por el cual puede pasar un solo auto a la vez. Un auto pide permiso para pasar por el puente, cruza por el mismo y luego sigue su camino.**

```

Monitor Puente
cond cola;
int cant= 0;

Procedure entrarPuente ()
    while ( cant > 0) wait (cola);
    cant = cant + 1;
end;

Procedure salirPuente ()
    cant = cant - 1;
    signal(cola);
end;
End Monitor;

Process Auto [a:1..M]
    Puente. entrarPuente ();
    "el auto cruza el puente"
    Puente. salirPuente();
End Process;

```

1. **¿El código funciona correctamente? Justifique su respuesta.**

-Hay busywaiting en entrarPuente(), hay que cambiar el while por un if

2. **¿Se podría simplificar el programa? ¿Sin monitor? ¿Menos procedimientos? ¿Sin variable condition? En caso afirmativo, rescriba el código.**

-Con semaforos se hace facil

```

sem mutex = 1;
Process Auto[ id:0..M-1]{
    P(mutex)
    V(mutex)
}

```

3. **¿La solución original respeta el orden de llegada de los vehículos? Si rescribió el código en el punto b), ¿esa solución respeta el orden de llegada?**

-No respetan el orden de llegada, ya que la variable cola es de condition y no una cola donde se pushean y popean los que van llegando en roden

2. Existen N procesos que deben leer información de una base de datos administrada por un motor que admite un número limitado de consultas simultáneas.
- Analice el problema y defina qué procesos, recursos y monitores/sincronizaciones serán necesarios/convenientes para resolverlo.
 - Implemente el acceso a la base de datos por parte de los procesos, sabiendo que el motor de la base de datos puede atender a lo sumo 5 consultas de lectura simultáneas.

```
Process Lector [id:0..N-1]{
    motor.Entrar()
    // Leo info
    motor.Salir()
}

Monitor motor {
    cond espera;
    int leyendo, esperando = 0;

    Procedure Entrar(){
        if (leyendo < 5) {
            leyendo++;
        }
        else {
            esperando++;
            wait(espera)
        }
    }

    Procedure Salir(){
        if (esperando > 0){
            esperando--;
            signal(espera)
        }
        else { leyendo--; }
    }
}
```

```
}  
}
```

3. Existen N personas que deben fotocopiar un documento. La fotocopidora sólo puede ser usada por una persona a la vez. Analice el problema y defina qué procesos, recursos y monitores serán necesarios/convenientes, además de las posibles sincronizaciones requeridas para resolver el problema. Luego, resuelva considerando las siguientes situaciones:

- a. Implemente una solución suponiendo que no importa el orden de uso. Existe una función Fotocopiar() que simula el uso de la fotocopidora.

```
Process Persona[id:0..N-1]{  
    impresora.imprimir()  
}  
  
Monitor Impresora{  
    cond espera;  
  
    Procedure imprimir(){  
        fotocopiar();  
    }  
}
```

- b. Modifique la solución de (a) para el caso en que se deba respetar el orden de llegada.

```
Process Persona[id:0..N-1]{  
    impresora.entrar()  
    fotocopiar()  
    impresora.salir()  
}  
  
Monitor Impresora{  
    cond espera[N];  
    bool libre = true;
```

```

int esperando = 0;

Procedure entrar(){
    if (not libre) {
        esperando++;
        wait(espera);
    } else {
        libre = false;
    }
}

Procedure salir(){
    if (esperando > 0) {
        esperando--;
        signal(espera)
    } else {
        libre = true;
    }
}
}

```

- c. Modifique la solución de (b) para el caso en que se deba dar prioridad de acuerdo con la edad de cada persona (cuando la fotocopidora está libre, la debe usar la persona de mayor edad entre las que estén esperando para usarla).

```

Process Persona[id:0..N-1]{
    int edad = random()
    Impresora.entrar(id,edad)
    fotocopiar()
    Impresora.salir()
}

Monitor Impresora{
    Cola c;
    cond espera[N];
    bool libre = true;

    Procedure entrar(int id,edad in){
        if (not libre) {

```

```

        insertar(c,id,edad) // Inserta ordenado p
or edad
        wait(espera[id])
    } else {
        libre = false;
    }

    Procedure salir(){
        int id;
        if (c.notEmpty()) {
            sacar(c,id)
            signal(espera[id])
        } else {
            libre = true;
        }
    }
}

```

- d. **Modifique la solución de (a) para el caso en que se deba respetar estrictamente el orden dado por el identificador del proceso (la persona X no puede usar la fotocopiadora hasta que no haya terminado de usarla la persona X-1).**

```

Process Persona[id:0..N-1]{
    Impresora.entrar(id)
    fotocopiar()
    Impresora.salir()
}

Monitor Impresora{
    cond espera[N];
    int siguiente = 0;

    Procedure entrar(int id in){
        if (id != siguiente){
            wait(espera[id]);
        }
    }

    Procedure salir(){

```

```

        siguiente++;
        if (siguiente < N) { signal(espera[sigue
nte]) }
    }
}

```

- e. Modifique la solución de (b) para el caso en que además haya un Empleado que le indica a cada persona cuándo debe usar la fotocopidora.

```

Process Persona[id:0..N-1]{
    Impresora.entrar()
    fotocopiar()
    Impresora.salir()
}

Process Empleado {
    while (true){
        impresora.sig()
    }
}

Monitor Impresora{
    cond espera, esperaLlegada, esperaSalida;
    int esperando = 0;

    Procedure entrar(){
        esperando++;
        signal(esperaLlegada);
        wait(espera);
    }

    Procedure salir(){ signal(esperaSalida); }

    Procedure sig() {
        if (esperando = 0) { wait(esperaLlegada); }
        cantEsperando--;
        signal(espera);
    }
}

```

```

        wait(esperaSalida);
    }
}

```

- f. **Modificar la solución (e) para el caso en que sean 10 fotocopadoras. El empleado le indica a la persona qué fotocopadora usar y cuándo hacerlo.**

```

Process Persona[id:0..N-1]{
    int idImp;
    Impresora.entrar(idImp,id)
    fotocopiar(idImp) // idImp lo modifican por parametro out
    Impresora.salir(idImp)
}

Process Empleado {
    while (true){
        impresora.atender();
    }
}

Monitor Impresora{
    cond espera[N], esperaLlegada, esperaSalida;
    Cola impresoras[10], c;
    int impresoraAUsar[N];

    Procedure entrar(imp: int out, id: int in){
        push(c,id); // Me encolo
        signal(esperaLlegada); // Aviso que llegue
        wait(espera[id]); // Espero que empleado me atienda
        idImp = impresoraAUsar[id]; // Uso impresora que me asigno Empleado
    }

    Procedure salir(idImp: int in){
        push(impresoras,idImp); // Pusheo impresora q

```

```

ue acabo de usar
    signal(esperaSalida); // Aviso que termine
}

Procedure atender(){
    int idImp, id;
    wait(esperaLlegada); // Espero que llegue
    if (impresoras.empty()){ wait(esperaSalida); }
// Si no hay libres espero
    idImp = pop(impresoras); // Saco el id de una
impresora y una persona
    id = pop(c);
    impresoraAUsar[id] = idImp; // Le pongo que i
mpresora usar
    signal(espera[id]); // Despierto para que la
use
}
}

```

4. Existen N vehículos que deben pasar por un puente de acuerdo con el orden de llegada. Considere que el puente no soporta más de 50000 kg y que cada vehículo cuenta con su propio peso (ningún vehículo supera el peso soportado por el puente).

```

// Preguntar si se puede usar c.top para no sacar de la
cola para ver el peso
Process Vehiculo [id: 0..N-1]{
    int peso = random();
    Puente.entrar(id,peso);
    Puente.salir(peso);
}

Monitor Puente{
    int pesoTotal = 0;
    Cola c[N];
    cond esperaA;
}

```

```

Procedure entrar(peso ,id: int in){
    if (!c.empty()) or (peso + pesoTotal > 50000){
        c.push(id,peso);
        wait(espera)
    } else {
        pesoPuede+=peso;
    }
}

Procedure salir (peso: in int){
    int siguiente, pesoSiguiente;
    pesoPuede -= peso;
    if (!c.empty()) {
        siguiente, pesoSiguiente = c.top() // Saco e
l id y peso del siguiente
        if (pesoPuede + pesoSiguiente <= 50000){
            // Si no supera el umbral, lo levanto
            pesoPuede+= pesoSiguiente;
            signal(espera);
        }
    }
}
}

```

5. En un corralón de materiales se debe atender a N clientes de acuerdo con el orden de llegada. Cuando un cliente es llamado para ser atendido, entrega una lista con los productos que comprará, y espera a que alguno de los empleados le entregue el comprobante de la compra realizada.

a. Resuelva considerando que el corralón tiene un único empleado.

```

Process Cliente[id:0..N-1]{
    text productos,comprobante;
    Corralon.llegue(id,lista,comprobante)
}

Process Empleado {
    text productos, comprobante;
    int id;
}

```

```

    for int i: 1..N {
        Corralon.atender(lista, id)
        crearComprobante(lista,id,comprobante);// Cre
o comp a partir de lista
        Corralon.enviarComp(id, comprobante)
    }
}

Monitor Corralon{
    cond espera[N];
    cond esperaEmpleado;
    Cola c;
    text comprobantes[N];

    Procedure llegue(id: in int, productos: in text,
comprobante: out text){
        push(c,id,productos);
        signal(esperaEmpleado);
        wait(espera[id]);
        comprobante = comprobantes[id];
    }

    Procedure atender(productos: in text, id: out in
t) {
        if (c.isEmpty()) {
            wait(esperaEmpleado);
        }
        pop(c,id,productos);
    }

    Procedure enviarComp(id: in int, comprobante: in
text){
        comprobantes[id] = comprobante;
        signal(espera[id]);
    }
}

```

- b. Resuelva considerando que el corralón tiene E empleados ($E > 1$). Los empleados no deben terminar su ejecución.
- c. Modifique la solución (b) considerando que los empleados deben terminar su ejecución cuando se hayan atendido todos los clientes.

```
// B y C
Process Cliente[id:0..N-1]{
    text productos,comprobante;
    Corralon.llegue(id,lista,comprobante)
}

Process Empleado [id:00..M-1] {
    text productos, comprobante;
    int id;
    while (true) {
        Corralon.atender(lista, id)
        crearComprobante(lista,id,comprobante);// Cre
o comp a partir de lista
        Corralon.enviarComp(id, comprobante)
    }
    /* modificacion inciso C
    for int i: 1..N {
        Corralon.atender(lista, id)
        crearComprobante(lista,id,comprobante);
        Corralon.enviarComp(id, comprobante)
    }
    */
}

Monitor Corralon{
    cond espera[N];
    cond esperaEmpleado;
    Cola c;
    text comprobantes[N];

    Procedure llegue(id: in int, productos: in text,
comprobante: out text){
        push(c,id,productos);
    }
}
```

```

        signal(esperaEmpleado);
        wait(espera[id]);
        comprobante = comprobantes[id];
    }

    Procedure atender(productos: in text, id: out in
t) {
        // Solo cambia if por while para no popear cu
        ando no se debe
        while (c.isEmpty()) {
            wait(esperaEmpleado);
        }
        pop(c,id,productos);
    }

    Procedure enviarComp(id: in int, comprobante: in
text){
        comprobantes[id] = comprobante;
        signal(espera[id]);
    }
}

```

6. Existe una comisión de 50 alumnos que deben realizar tareas de a pares, las cuales son corregidas por un JTP. Cuando los alumnos llegan, forman una fila. Una vez que están todos en fila, el JTP les asigna un número de grupo a cada uno. Para ello, suponga que existe una función **AsignarNroGrupo()** que retorna un número "aleatorio" del 1 al 25. Cuando un alumno ha recibido su número de grupo, comienza a realizar su tarea. Al terminarla, el alumno le avisa al JTP y espera por su nota. Cuando los dos alumnos del grupo completaron la tarea, el JTP les asigna un puntaje (el primer grupo en terminar tendrá como nota 25, el segundo 24, y así sucesivamente hasta el último que tendrá nota 1). Nota: el JTP no guarda el número de grupo que le asigna a cada alumno.

```

Process Alumno [id:0..49]{
    int grupo, nota;
    tarea.llegue(id,grupo);
}

```

```

    // hace tarea
    tarea.entregar(grupo, nota);
}

Process JTP{
    tarea.asignarGrupo();
    tarea.asignarPuntaje();
}

Monitor tarea{
    Cola c, entregas;
    cond esperaAlum[50], esperaJTP, esperaGrupo[25], avisoEntrega
    int cant = 0, puntaje = 25;
    int grupoAlumno[50], notaGrupo[25], finGrupo[25](25, 0);

    Procedure llegue(id: in int, grupo: out int){
        push(c, id); // Me encolo
        cant++; // Sumo 1 al contador
        if (cant == 50) { // Si estamos todos despierto
al JTP
            signal(esperaJTP);
        }
        wait(esperaAlum[id]); // Espero asignacion de grupo
        grupo = grupoAlumno[id]; // Agarro asignacion de grupo
    }

    Procedure entregar(grupo: in int, nota: out int){
        finGrupo[id]++;
        if (finGrupo[id] == 2) { // Si terminaron los 2
del grupo despierto a JTP
            push(entregas, grupo); // Pushea el 2do la tarea
            signal(avisoEntrega); // Aviso a JTP
            wait(esperaGrupo[grupo]); // Espero puntaje

```

```

    } else {wait(esperaGrupo[grupo]);} // Si soy el p
rimero espero a mi compa

    nota = notaGrupo[grupo]; // Me agarro el puntaje
}

Procedure asignarGrupo(){
    if (cant < 50) { wait(esperaJTP)};
    for i:1..50 {
        int id = pop(c); // Saco uno random
        grupoAlumno[id] = AsignarNroGrupo(); // Le d
oy un grupo
        signal(esperaAlu[id]); // Despierto a ese al
umno
    }
}

Procedure asignarPuntaje(){
    for i:1..50{
        if (entregas.empty()){ wait(avisoEntrega);}
        int grupo = pop(entregas); // Saco un grupo
que termino
        // Pongo nota y despierto al grupo
        notaGrupo[grupo] = puntaje;
        puntaje--;
        signal(esperaGrupo[grupo]);
        signal(esperaGrupo[grupo]);
    }
}
}

```

7. Se debe simular una maratón con C corredores donde en la llegada hay UNA máquina expendedora de agua con capacidad para 20 botellas. Además, existe un repositor encargado de reponer las botellas de la máquina. Cuando los C corredores han llegado al inicio, comienza la carrera. Cuando un corredor termina la carrera, se dirige a la máquina expendedora, espera su turno (respetando el orden de llegada), saca una

botella y se retira. Si encuentra la máquina sin botellas, le avisa al repositor para que cargue nuevamente la máquina con 20 botellas; espera a que se haga la recarga; saca una botella y se retira. Nota: mientras se reponen las botellas, se debe permitir que otros corredores se encolen.

```

Process Corredor[i:1..C-1]{
    maraton.llegue();
    //corro
    maraton.fin();
}

Process Repositor{
    while(true){
        maraton.esperaVacía();
        //repongo
        maraton.finReponer();
    }
}

Monitor maraton{
    cond esperaC, avisoReposicion, esperaMaquina[C];
    Cola c;
    int cant = 0, botellas = 20;
    bool reponiendo = false;

    Procedure llegue(){
        cant++; // Llego
        if (cant == C) { signal_all(esperaC)} // Si estamos todos los despierto
        else { wait(esperaC);} // Sino me duermo hasta que estemos todos
    }

    Procedure fin(){
        // Si no hay botellas o hay fila, espero reposicion o turno
        if (botellas == 0) or (!c.empty()){
            if (botellas == 0) AND (!reponiendo){//Si no

```

```

hay botellas y no avisaron,aviso
    reponiendo = true;
    signal(avisoReposicion); // Despierto re
positor para que reponga
}
push(c,id);
wait(esperaMaquina[id];
}
// Hay botellas y me despertaron
botellas--;
if (botellas > 0 AND !c.empty()){ // Si hay bote
llas y hay gente esperando
    int idProx = c.pop(); // Agarro y despierto
al siguiente
    signal(esperaMaquina[idProx];
}
}

Procedure esperaVacía(){
    if (!reponiendo){ wait(avisoReposicion);} // Si
no tengo que reponer duermo
}

Procedure finReponer(){
    cant = 20; // Reset cant botellas
    reponiendo = false; // Reset estado de reposicio
n
    if (!c.empty()){
        int idProx = c.pop();
        signal(esperaC[idProx]); // Despierto al que
me llamo para que reponga
    }
}
}

```

8. En un entrenamiento de fútbol hay 20 jugadores que forman 4 equipos (cada jugador conoce el equipo al cual pertenece llamando a la función DarEquipo()). Cuando un equipo está listo (han llegado los 5 jugadores

que lo componen), debe enfrentarse a otro equipo que también esté listo (los dos primeros equipos en juntarse juegan en la cancha 1, y los otros dos equipos juegan en la cancha 2). Una vez que el equipo conoce la cancha en la que juega, sus jugadores se dirigen a ella. Cuando los 10 jugadores del partido llegan a la cancha, comienza el partido; juegan durante 50 minutos y, al terminar, todos los jugadores del partido se retiran (no es necesario que esperen para salir).

9. En un examen de la secundaria hay un preceptor y una profesora que deben tomar un examen escrito a 45 alumnos. El preceptor se encarga de darles el enunciado del examen a los alumnos cuando los 45 han llegado (es el mismo enunciado para todos). La profesora se encarga de ir corrigiendo los exámenes de acuerdo con el orden en que los alumnos van entregando. Cada alumno, al llegar, espera a que le den el enunciado, resuelve el examen y, al terminar, lo deja para que la profesora lo corrija y le envíe la nota. Nota: maximizar la concurrencia; todos los procesos deben terminar su ejecución; suponga que la profesora tiene una función `corregirExamen` que recibe un examen y devuelve un entero con la nota.
10. En un parque hay un juego para ser usado por N personas de a una a la vez y de acuerdo al orden en que llegan para solicitar su uso. Además, hay un empleado encargado de desinfectar el juego durante 10 minutos antes de que una persona lo use. Cada persona, al llegar, espera hasta que el empleado le avisa que puede usar el juego, lo usa por un tiempo y luego lo devuelve. Nota: suponga que la persona tiene una función `Usar_juego` que simula el uso del juego; y el empleado tiene una función `Desinfectar_Juego` que simula su trabajo. Todos los procesos deben terminar su ejecución.